

Reviewer Pack — Minimal Rank of Frobenius Norm over XOR-AND Tree Width

Entry 7da7f6235073 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 12:04:57 UTC

Minimal Rank of Frobenius Norm over XOR-AND Tree Width

Entry ID: 7da7f6235073 Verdict: INCONCLUSIVE Recorded: 2026-05-27 12:04:57 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Frobenius Norm) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For any Boolean function f with n inputs, the XOR-AND tree width of f is upper bounded by $O(n^{1/3} * \log^n(f))$ where $\log^n(f) = \log(\log \dots \log(f))$, and the minimal rank of its associated Frobenius Norm is at most 2.’, ‘Equivalently, for all Boolean functions with XOR-AND tree width w , the minimal rank of the Frobenius Norm of their characteristic polynomial is at most $O(w^{1/3} * \log^w(f))$ ’, ‘where $\log^w(f) = \log(\log \dots \log(f))$ and f represents the function as a matrix under the Fourier transform.’]

Rationale (proposer’s reasoning):

[‘The Frobenius norm provides a measure of the complexity of a linear transformation, which can be related to the structure of XOR-AND trees that represent Boolean functions. This conjecture aims to connect the geometric properties of these norms with the computational complexity of the functions.’, ‘A low rank in the Frobenius Norm suggests that the function can be represented by a simpler linear transformation, potentially leading to more efficient algorithms for computing the tree width.’, ‘This connection could expose new insights into both representation theory and computational complexity.’]

Taxonomy category: Representation Theory, XOR-AND Tree Width (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): belacde319321e3c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of the minimal ranks of the Frobenius Norms, for all tested Boolean functions, is less than or equal to $O(w^{1/3} * \log^w(f))$ with at least 95% confidence level. The conjecture is falsified if this mean exceeds $O(w^{1/3} * \log^w(f))$ by more than 5 standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - frobenius norm AND representation theory AND XOR-AND tree width - minimal rank characteristic polynomial AND Frobenius Norm AND Boolean functions - Fourier transform matrix Boolean functions AND XOR-AND tree width AND $\log^n(\log \dots \log(f))$

Top relevant hits considered: - [<http://arxiv.org/abs/2603.27466v1>] English translation of Frobenius' and Stickelberger's "On the theory of elliptic functions" - [<http://arxiv.org/abs/2412.13909v2>] Graded Frobenius Algebras - [<http://arxiv.org/abs/2305.13803v1>] NORM: Knowledge Distillation via N-to-One Representation Matching - [<http://arxiv.org/abs/2308.14302v2>] Finite simple characteristic quotients of the free group of rank 2 - [<http://arxiv.org/abs/1106.3953v3>] On the characteristic polynomial of the Frobenius on étale cohomology - [<http://arxiv.org/abs/0804.1091v2>] Extensions of the Frobenius to ring of differential operators on polynomial algebra in prime characteristic - [<http://arxiv.org/abs/2105.13085v3>] Constraints on dark photon dark matter using data from LIGO's and Virgo's third observing run - [<http://arxiv.org/abs/2501.11635v2>] Search for charge-parity violation in semileptonically tagged $D^0 \rightarrow K^+ \pi^-$ decays

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def xor_and_tree_width(n):
        if n == 1:
            return 1
        else:
            return 2 * xor_and_tree_width(n - 1)

    def frobenius_norm(matrix):
        sum_of_squares = sum(sum(x**2 for x in row) for row in matrix)
        return math.sqrt(sum_of_squares)

    def characteristic_polynomial(matrix):
        n = len(matrix)
        if n == 1:
```

```

        return [matrix[0][0]]
    else:
        det = 0
        for j in range(n):
            sub_matrix = [[matrix[i][k] for k in range(n) if k != j] for i in range(1, n)]
            det += (-1)**j * matrix[0][j] * characteristic_polynomial(sub_matrix)
        return [det]

def min_rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for r in range(rows):
        if any(matrix[r][c] != 0 for c in range(cols)):
            rank += 1
            for c in range(cols):
                matrix[r][c] /= matrix[r][r]
            for i in range(rows):
                if i != r:
                    factor = matrix[i][r]
                    for j in range(cols):
                        matrix[i][j] -= factor * matrix[r][j]
    return rank

def log_n(f, n):
    result = f
    for _ in range(n):
        result = math.log(result)
    return result

n = random.randint(5, 40)
f = random.random()
w = xor_and_tree_width(n)
matrix = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

char_poly = characteristic_polynomial(matrix)
frob_norm = frobenius_norm(char_poly)
min_rank_val = min_rank(char_poly)

log_w_f = log_n(f, w)
upper_bound = w**(1/3) * log_w_f

return {
    "metric_name": "minimal_rank",
    "metric_value": min_rank_val,
    "instances_tested": 1,
    "conjecture_holds": min_rank_val <= upper_bound,
    "counterexample": "" if min_rank_val <= upper_bound else f"Counterexample: n={n}, w={w}, n"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.95:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=budget_exceeded n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ceda3078.py", line 89, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ceda3078.py", line 68, in run_trial
    char_poly = characteristic_polynomial(matrix)
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ceda3078.py", line 39, in characteristic_polynomial
    det += (-1)**j * matrix[0][j] * characteristic_polynomial(sub_matrix)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ceda3078.py", line 39, in characteristic_polynomial
    det += (-1)**j * matrix[0][j] * characteristic_polynomial(sub_matrix)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ceda3078.py", line 39, in characteristic_polynomial
    det += (-1)**j * matrix[0][j] * characteristic_polynomial(sub_matrix)
              ~~~~~
[Previous line repeated 29 more times]
TypeError: unsupported operand type(s) for +=: 'int' and 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution before producing data to evaluate the conjecture.
| next: Investigate and fix the error in the test code that caused it to crash, then rerun the test to gather data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12340
2	preregistration	ollama_remote	glm4:latest	0	0	6063
3	novelty	ollama_remote	glm4:latest	0	0	4693
4	novelty	ollama_remote	glm4:latest	0	0	6239
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16071
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18440
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12797
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10505
9	judge	ollama_remote	glm4:latest	0	0	20710

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 107858 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/7da7f6235073.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/7da7f6235073.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/7da7f6235073.tar.gz* (if generated)